Milestone 1: Technical Proposal

NCF + ALS Movie Recommender · Solo Team · MLOps-First Architecture

1. Problem Framing & User-Facing API Contract

A **real-time movie recommendation service** on MovieLens-1M (6K users × 3.7K movies, ~1M ratings), ingesting watch/rating events via managed Kafka, training two models (ALS baseline + Neural Collaborative Filtering), and serving top-K recommendations through a containerized FastAPI on Google Cloud Run. Emphasis on **production-grade MLOps**: automated retraining, versioning, A/B experimentation, schema validation, and full prediction provenance.

API Endpoints

Endpoint	Method	Description	SLO
/recommend/{user_id}?k=20&model=	GET	Top-K movie IDs (CSV); optional model selector for A/B	p95 < 200ms
/healthz	GET	Liveness probe: status + model version	p99 < 50ms
/metrics	GET	Prometheus (latency histogram, counters, errors)	Always up
/switch?model=vX.Y	POST	Hot-swap active model version (token-gated)	< 5s swap

Service Level Objectives

SLO	Target	Measurement
Availability	≥ 70% (M4: 72h pre + 144h post deadline)	GH Actions cron uptime probe
Latency (p95)	< 200ms for /recommend	Prometheus histogram quantile
Error rate	< 5%	status=500 counter / total
Model freshness	≤ 7 days since last retrain	model_registry metadata timestamp

2. Architecture Overview

See **Figure 1** (diagram page). Modular pipeline: **Ingest** → **Store** → **Train** → **Serve** → **Monitor** → **Experiment**. Kafka topics ({team}.watch, .rate, .reco_requests, .reco_responses) carry event streams. The ingestor validates schemas (pandera + Confluent Schema Registry, BACKWARD compat) and writes versioned Parquet to Cloudflare R2. Training reads snapshots, trains ALS + NCF, evaluates HR@10/NDCG@10 via chrono split, publishes artifacts + metadata to model registry. FastAPI loads at startup; /switch for hot-swap. GH Actions cron probe writes reco events to Kafka for online KPI (watch-after-reco rate, z-test).

3. Cloud & Tooling Choices

Component	Choice	Rationale
Streaming	Confluent Cloud (free)	Managed Kafka + Schema Registry; student credits
Compute	Google Cloud Run	Scale-to-zero; Docker-native; secret injection
Storage	Cloudflare R2	S3-compatible; free egress; versioned paths
Registry	ghcr.io	Native GH Actions integration; free public repos
CI/CD	GitHub Actions	Secrets/envs; cron for probes + retrain
Monitoring	Grafana Cloud (free)	Prometheus-compatible dashboards
ML	PyTorch (NCF) + implicit (ALS)	Flexibility + fast ALS baseline
Validation	pandera + Schema Registry	Dual-layer: Python + broker-side

4. Data Plan

Batch: MovieLens-1M as versioned Parquet under /snapshots/ml-1m/YYYY-MM-DD/. Chronological 80/10/10 train/val/test split by timestamp (no leakage). **Streaming:** Kafka consumer → Avro validate → rolling Parquet partitions + optional Redis cache. **Replay:** Offset 0 + idempotent writes (event ID dedup) for cold-start recovery.

5. MLOps Plan

5.1 Models

ALS Baseline — implicit library. ~2 min train, ~5MB, strong NDCG@10 baseline. A/B control arm. **NCF (NeuMF)** — GMF + MLP in PyTorch. Embed=64, MLP=[128,64,32], BCE+Adam, dropout=0.3. Target: beat ALS on ranking metrics, <50ms inference.

5.2 CI/CD Pipeline

On push to main: **Lint** (ruff + mypy) → **Test** (pytest ≥70% cov) → **Build** (multi-stage Docker, python:3.11-slim ~250MB) → **Push** (ghcr.io, digest in summary) → **Deploy** (gcloud run deploy). Secrets in GH Environments.

5.3 Registry & Retrain

Model artifacts at /model_registry/vX.Y/ with metadata.json [name, version, snapshot, date, metrics {HR@10, NDCG@10}, git_sha, duration, size]. Daily cron retrain → eval gate → promote if better → /switch. Previous version kept for <5s rollback.

5.4 Testing

Unit: data validation, feature eng, API contract. Integration: Kafka → Parquet round-trip. Smoke: valid output shape, no NaNs. Target: ≥70% coverage on non-ML code.

6. Solo Milestone Gantt

Tasks sequenced to minimize context-switching. High-risk items (Kafka, Cloud Run) front-loaded. 2-day buffer before each deadline.

MS	Wk	Focus Area	Key Deliverables
M1	1–2	Foundation: proposal, repo skeleton, arch diagram	This report + repo + project board
M2	3–4	Core: Kafka → ingest → train → deploy → probes	2 models, Cloud Run live, probes
M3	5–6	Quality: eval, CI/CD, tests, schema/drift	Eval reports, ≥70% cov, drift charts
M4	7–8	Ops: monitoring, retrain, A/B, provenance	Grafana, 2 updates, z-test results
M5	9–10	Responsible ML: fairness, security, demo	Report + video + slide deck

7. Risk Register (Top 5)

#	Risk	Impact	Prob.	Mitigation
R1	Confluent free-tier quota exhaustion	High	Med	Monitor daily; Redpanda backup; batch off-peak
R2	Cloud Run cold starts breach p95 SLO	High	Med	min-instances=1; small image ~250MB; cron pre-warm
R3	NCF overfits on MovieLens-1M	Med	Med	Early stop + dropout 0.3; 30 epoch cap; ALS fallback
R4	Solo burnout / scope creep	High	Med	Weekly scope freeze; 2-day buffer; cut optional first
R5	Schema evolution breaks consumer	Med	Low	Registry BACKWARD compat; pandera coerce; version all

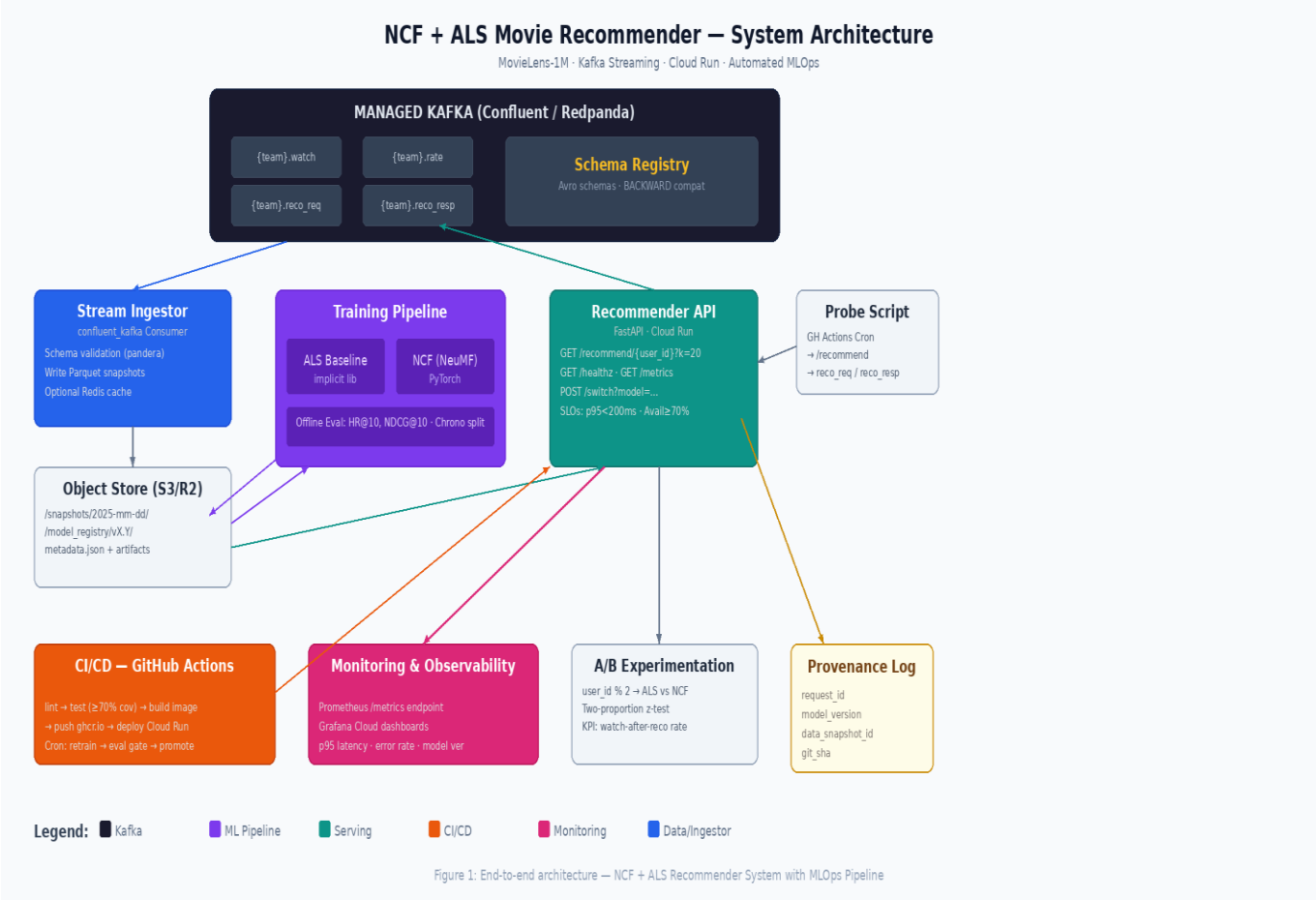
8. Repo Structure & Hygiene

```
recommender-system/
├── service/            # FastAPI app, model loading, Dockerfile
├── stream/             # Kafka consumer, schema validation, snapshot writer
├── training/           # train_als.py, train_ncf.py, evaluate.py
├── scripts/            # probe.py, retrain.py, switch_model.py
├── tests/              # Unit + integration tests
├── .github/workflows/  # CI/CD, cron probe, retrain workflows
├── monitoring/         # Grafana dashboard JSON, alert rules
├── docs/               # Architecture diagrams, runbooks
├── requirements.txt
├── Dockerfile
└── README.md
```

Labels: kafka, ml-model, infra, monitoring, fairness, bug, docs. **Board:** GitHub Projects Kanban (Backlog → In Progress → Review → Done). **README:** Overview, quickstart, API docs, setup guide.

Repo: https://github.com/sakethyalamanchili/movie-recommender-system · Contract: Appendix A · Diagram: Figure 1 (next page)

Figure 1: System Architecture Diagram



End-to-end lifecycle: Kafka ingestion → Parquet snapshots → dual-model training (ALS + NCF) → model registry → FastAPI on Cloud Run → Prometheus → Grafana → A/B (user_id % 2, z-test) → provenance. GH Actions cron probe drives the reco feedback loop for online KPIs. All automation designed for solo operation.

Appendix A: Team Contract (Solo)

Section	Details
Team Member	**Saketh Yalamanchili** — MS AIML student. Skills: PyTorch, deep learning, scikit-learn, Python, FastAPI, Docker, GitHub Actions, GCP Cloud Run, Kafka, Prometheus/Grafana. Prior experience: ML model deployment, REST API development.
Roles	Solo developer: PM, ML Lead, Data/Streaming Lead, DevOps/Cloud Lead. Sprint priority: (1) unblock deployment/infra, (2) ML training & eval, (3) monitoring, (4) docs. No backups needed — mitigated by thorough documentation and runbooks.
Communication	Self-managed via GitHub Issues + Project Board (Kanban). Daily git commits = standup log. Instructor office hours for blockers. Conflict resolution: N/A — decisions documented in PR descriptions.
Cadence	1-week sprints aligned to milestones. Sunday: plan issues. Friday: self-review + close issues. Demo prep: 48h before deadline. Sprint length: 7 days.
Expectations	Self-PR with description before merge to main. Done = tests pass + Docker builds + deploys + README updated. Min 70% coverage on glue code. Every PR documents changes and testing.
Decision Process	Model changes: compare eval_report.json metrics, promote only if NDCG@10 improves. Rollbacks: /switch (<5s). Arch changes: documented in Issue before implementation. Outages: GH Actions cron email alerts.
Risk & Contingency	Credits monitored weekly. Secrets in GH Environments only — rotate if leaked. Backups: Redpanda if Confluent quota hit; Fly.io if Cloud Run issues. On-call: cron probes + email; respond within 4h during windows.
Peer Accountability	Self-assessment (1–5) in each milestone PDF. Evidence: git history, GH Actions logs, Grafana uptime. Self-scored on: commit consistency, deadline adherence, doc quality, system availability.